BankCore

Full Implementation Plan

Core Banking Software — Microservices Architecture

Laravel 11 | PHP 8.3 | MySQL | Redis | Docker | Razorpay

8 Phases	30 Weeks	7 Services	1 Developer

Table of Contents

Section 1 — Project Overview & Goals
Section 2 — Monorepo Structure & Repository Setup
Section 3 — Tech Stack — Every Layer Explained
Section 4 — Microservices Architecture — The 7 Services
Section 5 — Database Design — Rules & Schemas
Section 6 — API Design Standards
Section 7 — Security & Compliance Implementation
Section 8 — Phase 0 — Foundation & Docker Setup
Section 9 — Phase 1 — Auth Service
Section 10 — Phase 2 — Customer Service & KYC
Section 11 — Phase 3 — Account Service
Section 12 — Phase 4 — Transaction Service
Section 13 — Phase 5 — Payment Service & Razorpay
Section 14 — Phase 6 — Loan Service
Section 15 — Phase 7 — Frontend (Blade + Livewire)
Section 16 — Phase 8 — Testing, Docs & Deployment
Section 17 — Timeline & Effort Breakdown
Section 18 — Decision Log
Section 19 — Learning Resources & Next Steps

Section 1 — Project Overview & Goals

What You Are Building

BankCore is a full-featured core banking web application built with a microservices architecture on Laravel 11 and PHP 8.3. It simulates a real Indian banking platform covering customer onboarding, KYC, bank accounts, transactions, loans, and Razorpay payment integration — all from scratch, solo, as a learning and portfolio project.

Three Goals — One Project

Goal	What It Means	How This Project Delivers
Deep Learning	Understand banking systems, microservices, security	Every service teaches a new real-world pattern
Portfolio	Impress fintech/bank employers or clients	Production-grade code, docs, deployed demo
Future Product	Could become a real NBFC/fintech tool	Compliance-ready design from day one

What This Plan Covers

- 8 build phases with clear start/end criteria
- 7 independent Laravel microservices with their own databases
- Full database schemas for every service
- API design standards with exact endpoint definitions
- Security implementation guide (encryption, audit trails, JWT)
- Razorpay full integration: pay-in, payouts, refunds, webhooks
- Loan lifecycle: application, approval, EMI, repayment, foreclosure
- KYC workflow simulation (no real Aadhaar API — sandbox only)
- Frontend with Blade + Livewire + TailwindCSS
- Testing strategy with PHPUnit/Pest
- Deployment guide on a cheap VPS (DigitalOcean / Railway)

Non-Goals (Explicitly Out of Scope)

- No real RBI license or live money — sandbox/test only
- No mobile app — web only
- No live Aadhaar/UIDAI API — simulated KYC approval flow
- No separate React/Vue frontend — Blade + Livewire keeps complexity manageable
- No Kafka or RabbitMQ — Laravel Queues over Redis is sufficient

Section 2 — Monorepo Structure & Repository Setup

All 7 services live in a single Git repository. This is called a monorepo. Each service is a fully independent Laravel application with its own database, its own .env file, and its own routes. They share nothing except the Git repository itself and the communication contracts (API contracts defined in docs/).

Why Monorepo for a Solo Developer

Approach	Pros	Cons
Monorepo (chosen)	One clone, one place to see everything, easy cross-service docs	Slightly more complex Git discipline
Polyrepo	Cleaner isolation per service	Need to clone 7 repos, hard to navigate solo
Monolith	Simplest to start	Hard to refactor later, defeats microservices learning goal

Full Directory Structure

```
bankcore/
├── services/
│   ├── api-gateway/          ← Laravel app: public entry point, JWT check, proxy
│   ├── auth-service/         ← Laravel app: register, login, tokens
│   ├── customer-service/     ← Laravel app: profiles, KYC docs, status
│   ├── account-service/      ← Laravel app: bank accounts, balances
│   ├── transaction-service/  ← Laravel app: deposits, withdrawals, transfers
│   ├── loan-service/         ← Laravel app: applications, EMI, repayment
│   └── payment-service/      ← Laravel app: Razorpay full integration
├── infrastructure/
│   ├── docker-compose.yml    ← boots all 7 services + MySQL + Redis
│   ├── nginx/
│   │   └── default.conf      ← routes /api/accounts/* → account-service etc.
│   └── mysql/
│       └── init.sql          ← creates all 7 databases on first boot
├── docs/
│   ├── architecture.md       ← system overview diagram description
│   ├── api-contracts/        ← one .md file per service with all endpoints
│   ├── db-schemas/           ← one .md file per service with schema
│   └── decisions.md          ← decision log (what was decided and why)
├── .gitignore
└── README.md
```

Git Setup Commands

```
mkdir bankcore && cd bankcore
```

```
git init
mkdir -p services infrastructure/nginx infrastructure/mysql docs/api-contracts
docs/db-schemas

# Create each Laravel service
cd services
laravel new api-gateway --no-interaction
laravel new auth-service --no-interaction
laravel new customer-service --no-interaction
laravel new account-service --no-interaction
laravel new transaction-service --no-interaction
laravel new loan-service --no-interaction
laravel new payment-service --no-interaction

# Go back to root and make first commit
cd ../
git add .
git commit -m "chore: initial monorepo scaffold with 7 service skeletons"
```

Section 3 — Tech Stack

Layer	Technology	Version	Why
Backend	Laravel	11.x	You know it — auth, queues, ORM, validation all built in
Language	PHP	8.3	Enums, readonly props, typed class constants
Database	MySQL	8.0	Reliable, relational, familiar — one instance per service
Cache/Queue	Redis	7.x	Fast queues between services, session cache
Auth	Laravel Sanctum + JWT	latest	Stateless tokens, works across all services
Payments	Razorpay PHP SDK	latest	Best sandbox, official PHP SDK, all features
Frontend	Blade + Livewire	3.x	No context switch — stay in PHP/Laravel world
CSS	TailwindCSS	3.x	Utility-first — fast professional UI without a designer
Local Dev	Docker + Compose	latest	One command boots all 7 services
API Docs	Scramble	latest	Auto-generates OpenAPI docs from Laravel code
Testing	Pest	2.x	Clean syntax, Laravel native support
Web Server	nginx	latest	Reverse proxy routing to each service

Key PHP 8.3 Features You Will Use

- Enums — for transaction types, account status, KYC status, loan states
- Readonly properties — for immutable transaction value objects
- Match expressions — for routing payment webhook event types
- Constructor property promotion — cleaner DTOs and service classes
- Union types — for flexible API response handling

Section 4 — Microservices Architecture — The 7 Services

Each service is described with: its single responsibility, what data it owns, what other services it calls, and its internal port number in Docker.

Service Communication Rules

The Golden Rules of Microservices

Rule 1: A service NEVER directly queries another service database. It calls the API.
Rule 2: Synchronous calls (HTTP) for user-facing reads. Async (queue events) for side-effects.
Rule 3: Each service handles its own errors. Never let one service crash cascade to another.
Rule 4: Services communicate via internal Docker network — never exposed publicly except API Gateway.

Service Map

Service	Port	Responsibility	Calls
api-gateway	8000	Public entry point, JWT validation, request proxying	All services
auth-service	8001	Register, login, token issue/refresh/revoke	None
customer-service	8002	Customer profile, KYC docs & status workflow	auth-service (token verify)
account-service	8003	Bank accounts, account numbers, balance state	customer-service (KYC check)
transaction-service	8004	Deposits, withdrawals, transfers, ledger	account-service (balance)
loan-service	8005	Loan applications, approval, EMI, repayment	customer, account, transaction
payment-service	8006	Razorpay: orders, verify, refund, payout, webhook	transaction-service

Request Flow Example — Customer Makes a Deposit

```
Customer Browser
    → POST /api/v1/transactions/deposit  (with JWT token)
    → API Gateway (port 8000)
         validates JWT token (calls auth-service internally)
         proxies request → Transaction Service (port 8004)
    → Transaction Service
         checks account balance → GET account-service/api/v1/accounts/{id}/balance
         creates transaction record (immutable INSERT)
         updates balance → PATCH account-service/api/v1/accounts/{id}/balance
         fires event → Redis queue → "transaction.completed"
    → API Gateway returns 201 Created to browser
```

Section 5 — Database Design — Rules & Schemas

Universal Database Rules

1. Use UUID primary keys on all tables shared across services (not auto-increment integers)
2. Use DECIMAL(15,2) for all money — NEVER float or double
3. Transactions table is IMMUTABLE — no updates, no deletes, ever
4. Sensitive PII (Aadhaar, PAN) stored encrypted using Laravel encrypt()
5. SoftDeletes on customers, accounts — never hard delete financial data
6. Every sensitive table gets an audit_logs companion record
7. All timestamps in UTC, stored as DATETIME

Auth Service Database

```
-- users
id              CHAR(36) PK   -- UUID
email           VARCHAR(255)  UNIQUE NOT NULL
phone           VARCHAR(15)   UNIQUE NOT NULL
password        VARCHAR(255)  -- bcrypt hash
role            ENUM(admin, bank_officer, customer)  DEFAULT customer
is_active       TINYINT(1)    DEFAULT 1
email_verified  TINYINT(1)    DEFAULT 0
created_at      DATETIME
updated_at      DATETIME

-- personal_access_tokens (Laravel Sanctum)
id              BIGINT PK
tokenable_type  VARCHAR(255)
```

```
tokenable_id    CHAR(36)
name            VARCHAR(255)
token           VARCHAR(64)   UNIQUE
abilities       TEXT
last_used_at    DATETIME NULL
expires_at      DATETIME NULL
created_at      DATETIME
updated_at      DATETIME
```

Customer Service Database

```
-- customers
id              CHAR(36) PK   -- UUID
user_id         CHAR(36)      -- references auth-service users.id (not FK)
first_name      VARCHAR(100)
last_name       VARCHAR(100)
dob             DATE
gender          ENUM(male, female, other)
address_line1   VARCHAR(255)
address_line2   VARCHAR(255) NULL
city            VARCHAR(100)
state           VARCHAR(100)
pincode         CHAR(6)
pan_number      TEXT          -- ENCRYPTED with Laravel encrypt()
aadhaar_number  TEXT          -- ENCRYPTED (last 4 digits stored plain for
display)
aadhaar_last4   CHAR(4)
kyc_status      ENUM(pending, under_review, approved, rejected)  DEFAULT pending
kyc_remarks     TEXT NULL
kyc_approved_by CHAR(36) NULL -- officer user_id
kyc_approved_at DATETIME NULL
deleted_at      DATETIME NULL -- SoftDeletes
created_at      DATETIME
updated_at      DATETIME

-- kyc_documents
id              CHAR(36) PK
customer_id     CHAR(36)      INDEX
type            ENUM(pan_card, aadhaar_front, aadhaar_back, address_proof, photo)
file_path       VARCHAR(500)  -- relative path in storage/
file_name       VARCHAR(255)
mime_type       VARCHAR(100)
file_size       INT
is_verified     TINYINT(1) DEFAULT 0
uploaded_at     DATETIME
created_at      DATETIME

-- customer_audit_logs
id              BIGINT PK AUTO_INCREMENT
customer_id     CHAR(36)      INDEX
action          VARCHAR(100)  -- e.g. kyc_status_changed, profile_updated
old_value       JSON NULL
new_value       JSON NULL
```

```
performed_by    CHAR(36)      -- user_id
ip_address      VARCHAR(45)
created_at      DATETIME
```

Account Service Database

```
-- accounts
id              CHAR(36) PK
customer_id     CHAR(36)      INDEX  -- references customer-service
account_number  VARCHAR(20)   UNIQUE NOT NULL
type            ENUM(savings, current)
status          ENUM(active, frozen, dormant, closed)  DEFAULT active
balance         DECIMAL(15,2) DEFAULT 0.00
currency        CHAR(3)       DEFAULT INR
branch_code     VARCHAR(10)   DEFAULT HQ001
ifsc_code       VARCHAR(11)   DEFAULT BANK0001001
opened_at       DATETIME
closed_at       DATETIME NULL
deleted_at      DATETIME NULL
created_at      DATETIME
updated_at      DATETIME

-- Account number format: BANK + YYYY + 8-digit sequence
-- Example: BANK202600000001
```

Transaction Service Database

```
-- transactions  (IMMUTABLE — no UPDATE, no DELETE ever)
id              CHAR(36) PK
account_id      CHAR(36)      INDEX
type            ENUM(credit, debit)
amount          DECIMAL(15,2) NOT NULL
balance_after   DECIMAL(15,2) NOT NULL  -- snapshot at time of transaction
channel         ENUM(cash, upi, neft, imps, rtgs, internal, razorpay)
reference_no    VARCHAR(50)   UNIQUE    -- generated: TXN + timestamp + random
description     VARCHAR(500)
status          ENUM(pending, completed, failed, reversed)  DEFAULT pending
related_txn_id  CHAR(36) NULL           -- for transfers: links debit to credit
initiated_by    CHAR(36)                -- user_id from auth-service
metadata        JSON NULL               -- razorpay_order_id, webhook data, etc.
created_at      DATETIME
-- NO updated_at — transactions do not change

-- transaction_reversals (when a completed txn must be reversed)
id              CHAR(36) PK
original_txn_id CHAR(36)      INDEX
reversal_txn_id CHAR(36)
reason          TEXT
reversed_by     CHAR(36)
```

```
created_at      DATETIME
```

Loan Service Database

```
-- loan_applications
id              CHAR(36) PK
customer_id     CHAR(36)      INDEX
account_id      CHAR(36)      -- disbursement account
type            ENUM(personal, home, vehicle, education, business)
amount          DECIMAL(15,2)
tenure_months   INT
interest_rate   DECIMAL(5,2)  -- annual percentage
purpose         TEXT
status          ENUM(draft, submitted, under_review, approved, rejected,
disbursed, closed)
reviewed_by     CHAR(36) NULL
reviewed_at     DATETIME NULL
rejection_reason TEXT NULL
disbursed_at    DATETIME NULL
disbursement_txn_id CHAR(36) NULL
created_at      DATETIME
updated_at      DATETIME

-- loan_emi_schedules
id              CHAR(36) PK
loan_id         CHAR(36)      INDEX
emi_number      INT
due_date        DATE
emi_amount      DECIMAL(15,2)
principal       DECIMAL(15,2)
interest        DECIMAL(15,2)
outstanding     DECIMAL(15,2)
status          ENUM(upcoming, paid, overdue, waived)
paid_at         DATETIME NULL
payment_txn_id  CHAR(36) NULL
created_at      DATETIME
```

Payment Service Database

```
-- razorpay_orders
id              CHAR(36) PK
razorpay_order_id VARCHAR(100) UNIQUE
account_id      CHAR(36)
customer_id     CHAR(36)
amount          DECIMAL(15,2)
currency        CHAR(3)       DEFAULT INR
status          ENUM(created, attempted, paid, failed)
razorpay_payment_id VARCHAR(100) NULL
razorpay_signature  VARCHAR(255) NULL
metadata        JSON NULL
```

```
created_at      DATETIME
updated_at      DATETIME

-- webhook_logs  (audit every webhook received)
id              BIGINT PK AUTO_INCREMENT
event           VARCHAR(100)  -- e.g. payment.captured, refund.created
payload         JSON
signature_valid TINYINT(1)
processed       TINYINT(1)    DEFAULT 0
error           TEXT NULL
received_at     DATETIME
```

Section 6 — API Design Standards

Universal Response Format

Every service must return this exact structure. Create a BaseApiController with a success() and error() helper that every controller extends.

```
// Success Response
{
  "success": true,
  "data": { ... },
  "message": "Account created successfully",
  "meta": {
    "timestamp": "2026-03-30T10:00:00Z",
    "version": "v1",
    "request_id": "uuid-here"
  }
}

// Error Response
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_BALANCE",
    "message": "Account does not have sufficient funds for this withdrawal",
    "details": { "available": 500.00, "requested": 1000.00 }
  },
  "meta": { "timestamp": "2026-03-30T10:00:00Z" }
}
```

Complete Endpoint Reference

Method	Endpoint	Service	Description
POST	/api/v1/auth/register	auth	Register new user

POST	/api/v1/auth/login	auth	Login, returns JWT
POST	/api/v1/auth/refresh	auth	Refresh access token
POST	/api/v1/auth/logout	auth	Revoke current token
GET	/api/v1/auth/me	auth	Get authenticated user
POST	/api/v1/customers	customer	Create customer profile
GET	/api/v1/customers/{id}	customer	Get customer profile
PUT	/api/v1/customers/{id}	customer	Update customer profile
POST	/api/v1/customers/{id}/kyc/documents	customer	Upload KYC document
PATCH	/api/v1/customers/{id}/kyc/status	customer	Officer: approve/reject KYC
GET	/api/v1/customers/{id}/kyc/status	customer	Get KYC status
POST	/api/v1/accounts	account	Open new bank account
GET	/api/v1/accounts/{id}	account	Get account details
GET	/api/v1/accounts/{id}/balance	account	Get current balance
PATCH	/api/v1/accounts/{id}/status	account	Freeze / close account
GET	/api/v1/accounts/customer/{customerId}	account	List customer accounts
POST	/api/v1/transactions/deposit	transaction	Deposit money
POST	/api/v1/transactions/withdraw	transaction	Withdraw money
POST	/api/v1/transactions/transfer	transaction	Transfer between accounts
GET	/api/v1/transactions/{accountId}/history	transaction	Paginated history
GET	/api/v1/transactions/{id}	transaction	Get single transaction
POST	/api/v1/loans/apply	loan	Submit loan application
GET	/api/v1/loans/{id}	loan	Get loan details
PATCH	/api/v1/loans/{id}/review	loan	Officer: approve/reject
POST	/api/v1/loans/{id}/disburse	loan	Disburse approved loan
GET	/api/v1/loans/{id}/emi-schedule	loan	Get EMI schedule
POST	/api/v1/loans/{id}/repay	loan	Record EMI repayment
POST	/api/v1/payments/create-order	payment	Create Razorpay order
POST	/api/v1/payments/verify	payment	Verify payment signature
POST	/api/v1/payments/refund	payment	Initiate refund
POST	/api/v1/payments/payout	payment	Razorpay payout
POST	/api/v1/payments/webhook	payment	Razorpay webhook receiver

Section 7 — Security & Compliance Implementation

Authentication Flow

```
1. User POSTs /api/v1/auth/login with email + password
2. Auth Service validates credentials, creates Sanctum token
3. Returns: { access_token, refresh_token, expires_in: 900 }  // 15 min
4. Client stores token, sends in every request:
     Authorization: Bearer <token>
5. API Gateway calls auth-service internally to verify token on every request
6. If expired: client uses refresh_token to get new access_token (no re-login)
7. On logout: token is revoked in personal_access_tokens table
```

Encryption Implementation

```
// In Customer model — encrypt PAN and Aadhaar before saving
class Customer extends Model
{
    protected $casts = [
        'pan_number'     => EncryptedCast::class,
        'aadhaar_number' => EncryptedCast::class,
    ];

    // Store only last 4 digits of Aadhaar in plain for display
    public function setAadhaarNumberAttribute(string $value): void
    {
        $this->attributes['aadhaar_number'] = encrypt($value);
        $this->attributes['aadhaar_last4']  = substr($value, -4);
    }
}
```

Audit Trail Trait

```
// app/Traits/Auditable.php  — apply to any model that needs audit logging
trait Auditable
{
    protected static function bootAuditable(): void
    {
        static::updating(function ($model) {
            AuditLog::create([
                'entity_type'  => class_basename($model),
                'entity_id'    => $model->id,
                'action'       => 'updated',
                'old_value'    => $model->getOriginal(),
                'new_value'    => $model->getDirty(),
                'performed_by' => auth()->id() ?? 'system',
                'ip_address'   => request()->ip(),
            ]);
```

```
        });
    }
}
```

Razorpay Webhook Security — Critical

```
// ALWAYS verify Razorpay webhook signature before processing
public function handleWebhook(Request $request): JsonResponse
{
    $payload   = $request->getContent();
    $signature = $request->header("X-Razorpay-Signature");
    $secret    = config("services.razorpay.webhook_secret");

    $expectedSignature = hash_hmac("sha256", $payload, $secret);

    if (! hash_equals($expectedSignature, $signature)) {
        WebhookLog::create(["event" => "unknown", "payload" => $payload,
                            "signature_valid" => false]);
        return response()->json(["error" => "Invalid signature"], 401);
    }

    // Now safe to process
    WebhookLog::create(["event" => $request->input("event"),
                        "payload" => $request->all(), "signature_valid" => true]);

    // Dispatch to queue — never process webhooks synchronously
    ProcessRazorpayWebhook::dispatch($request->all());

    return response()->json(["status" => "queued"], 200);
}
```

Security Checklist

Security Control	Where	Implementation	Priority
JWT Auth	API Gateway	Sanctum tokens, 15-min expiry	Critical
PAN/Aadhaar Encryption	customer-service	Laravel EncryptedCast	Critical
Webhook Signature	payment-service	HMAC SHA256 verify	Critical
Input Validation	All services	Laravel Form Requests	Critical
Audit Logging	All services	Auditable trait	High
Rate Limiting	API Gateway	Laravel throttle middleware	High
SQL Injection	All services	Eloquent ORM (no raw queries)	High
CORS	API Gateway	Laravel CORS config	Medium
HTTPS	nginx	SSL cert (Let's Encrypt on VPS)	High

Soft Deletes	customer, account	SoftDeletes trait	Medium

Section 8 — Phase 0: Foundation & Docker Setup

PHASE 0

Foundation — Docker, Nginx, Service Skeletons

Weeks 1–2

Goals

- All 7 Laravel services boot in Docker with one command
- nginx routes requests to the correct service
- Every service responds to GET /api/v1/health
- All 7 MySQL databases are created automatically
- Redis is running and accessible to all services
- You fully understand the infrastructure before writing any business logic

docker-compose.yml

```
version: "3.9"

services:
  nginx:
    image: nginx:alpine
    ports: ["8080:80"]
    volumes:
      - ./infrastructure/nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on: [api-gateway, auth-service, customer-service,
                 account-service, transaction-service, loan-service, payment-
service]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  mysql-auth:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_auth, MYSQL_ROOT_PASSWORD: secret }
    ports: ["3307:3306"]

  mysql-customer:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_customer, MYSQL_ROOT_PASSWORD: secret
}
    ports: ["3308:3306"]

  mysql-account:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_account, MYSQL_ROOT_PASSWORD: secret }
```

```
    ports: ["3309:3306"]

  mysql-transaction:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_transaction, MYSQL_ROOT_PASSWORD:
secret }
    ports: ["3310:3306"]

  mysql-loan:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_loan, MYSQL_ROOT_PASSWORD: secret }
    ports: ["3311:3306"]

  mysql-payment:
    image: mysql:8.0
    environment: { MYSQL_DATABASE: bankcore_payment, MYSQL_ROOT_PASSWORD: secret }
    ports: ["3312:3306"]

  api-gateway:
    build: ./services/api-gateway
    environment: { APP_PORT: 8000 }
    depends_on: [redis]

  auth-service:
    build: ./services/auth-service
    environment: { APP_PORT: 8001, DB_HOST: mysql-auth, DB_DATABASE: bankcore_auth
}
    depends_on: [mysql-auth, redis]

  # ... repeat for each service with correct mysql-* and port
```

nginx Routing Config

```
server {
  listen 80;

  location /api/v1/auth/ {
    proxy_pass http://auth-service:8001;
  }
  location /api/v1/customers/ {
    proxy_pass http://customer-service:8002;
  }
  location /api/v1/accounts/ {
    proxy_pass http://account-service:8003;
  }
  location /api/v1/transactions/ {
    proxy_pass http://transaction-service:8004;
  }
  location /api/v1/loans/ {
    proxy_pass http://loan-service:8005;
  }
  location /api/v1/payments/ {
```

```
    proxy_pass http://payment-service:8006;
  }
  # All other /api/v1/ goes to api-gateway
  location /api/ {
    proxy_pass http://api-gateway:8000;
  }
}
```

Phase 0 Done When

docker compose up starts without errors
curl http://localhost:8080/api/v1/health returns {"success":true} from API Gateway
curl http://localhost:8080/api/v1/auth/health returns {"success":true} from Auth Service
All 7 MySQL databases exist (verify with TablePlus or DBeaver)
Redis is reachable: docker exec -it bankcore-redis-1 redis-cli ping returns PONG

Section 9 — Phase 1: Auth Service

PHASE 1

Authentication — Register, Login, JWT Tokens

Weeks 3–4

What to Build

- User registration with email, phone, password, role
- Login returning access_token (15 min) and refresh_token (1 day)
- Refresh token endpoint
- Logout (revoke token)
- GET /me endpoint — returns authenticated user
- Middleware: VerifyInternalToken — used by all other services to validate tokens
- Database seeder: creates admin, bank_officer, and customer test users

Key Implementation

```
// routes/api.php
Route::prefix('v1/auth')->group(function () {
    Route::post('register', [AuthController::class, 'register']);
    Route::post('login',    [AuthController::class, 'login']);
    Route::post('refresh',  [AuthController::class, 'refresh']);
    Route::middleware('auth:sanctum')->group(function () {
        Route::post('logout', [AuthController::class, 'logout']);
        Route::get('me',      [AuthController::class, 'me']);
```

```
    });
});

// AuthController::login
public function login(LoginRequest $request): JsonResponse
{
    if (! Auth::attempt($request->only("email", "password"))) {
        return $this->error("INVALID_CREDENTIALS", "Email or password incorrect",
401);
    }

    $user         = Auth::user();
    $accessToken  = $user->createToken('access',  ['*'], now()->addMinutes(15));
    $refreshToken = $user->createToken('refresh', ['refresh'], now()->addDay());

    return $this->success([
        'access_token'  => $accessToken->plainTextToken,
        'refresh_token' => $refreshToken->plainTextToken,
        'expires_in'    => 900,
        'user'          => new UserResource($user),
    ], "Login successful");
}
```

Phase 1 Done When

POST /api/v1/auth/register creates a user and returns tokens
POST /api/v1/auth/login returns access_token + refresh_token
GET /api/v1/auth/me with valid token returns user data
GET /api/v1/auth/me with no token returns 401
POST /api/v1/auth/logout revokes the token (subsequent requests return 401)
Database has 3 seeded users: admin@bank.com, officer@bank.com, customer@bank.com

Section 10 — Phase 2: Customer Service & KYC

PHASE 2

Customer Profiles & KYC Workflow

Weeks 5–7

What to Build

- Customer profile creation (linked to auth user_id)
- Profile read and update
- KYC document upload (PAN card, Aadhaar front/back, photo, address proof)
- KYC status state machine: pending → under_review → approved/rejected
- PAN and Aadhaar stored encrypted, last 4 digits of Aadhaar stored plain

- Officer-only endpoint to approve or reject KYC with remarks
- Audit log on every KYC status change
- Cross-service: account-service will call this to verify KYC before opening account

KYC State Machine

```
// app/Enums/KycStatus.php
enum KycStatus: string
{
    case Pending     = 'pending';
    case UnderReview = 'under_review';
    case Approved    = 'approved';
    case Rejected    = 'rejected';

    public function canTransitionTo(self $next): bool
    {
        return match($this) {
            self::Pending     => $next === self::UnderReview,
            self::UnderReview => in_array($next, [self::Approved,
self::Rejected]),
            self::Rejected    => $next === self::Pending,  // allow resubmission
            self::Approved    => false,  // approved is terminal
        };
    }
}
```

File Upload Strategy

- Files stored in storage/app/kyc/{customer_id}/{document_type}/
- Use Laravel Storage facade — never expose direct file paths in API responses
- Generate a signed URL when an officer needs to view a document
- Validate: max 5MB, only jpg/png/pdf accepted, virus scan not needed for sandbox

Phase 2 Done When

Customer can create profile and upload all 5 KYC document types
KYC status starts as pending, moves to under_review when all docs uploaded
Officer can call PATCH /kyc/status to approve or reject with remarks
Approved customer: GET /customers/{id}/kyc/status returns { status: "approved" }
PAN number is stored encrypted — viewing DB directly shows only cipher text
Every status change creates an audit log entry

Section 11 — Phase 3: Account Service

PHASE 3

Bank Accounts — Open, Manage, Balance

Weeks 8–9

What to Build

- Open a bank account (savings or current)
- Guard: only KYC-approved customers can open accounts — calls customer-service API
- Account number generation: BANK + YYYY + 8-digit zero-padded sequence
- Get account details and balance
- List all accounts for a customer
- Account status management: freeze, unfreeze, close (officer only)

Cross-Service Call — KYC Check

```
// app/Services/CustomerVerificationService.php
class CustomerVerificationService
{
    public function __construct(private readonly Http $http) {}

    public function isKycApproved(string $customerId): bool
    {
        $response = Http::withToken(config('services.internal_token'))
            ->get(config('services.customer_service_url') . '/api/v1/customers/' .
$customerId . '/kyc/status');

        if ($response->failed()) {
            throw new ServiceUnavailableException("Customer service unreachable");
        }

        return $response->json('data.status') === 'approved';
    }
}
```

Account Number Generator

```
// app/Services/AccountNumberGenerator.php
class AccountNumberGenerator
{
    public function generate(): string
    {
        do {
            $number = 'BANK' . now()->format('Y') . str_pad(
                random_int(1, 99999999), 8, "0", STR_PAD_LEFT
            );
```

```
        } while (Account::where('account_number', $number)->exists());

        return $number;
    }
}
```

Phase 3 Done When

KYC-approved customer can open a savings or current account
Non-KYC customer gets 403 with code: KYC_NOT_APPROVED
Account number is unique and in format BANK20260000001
GET /accounts/{id}/balance returns current balance
Officer can freeze an account — frozen accounts reject transactions (tested in Phase 4)

Section 12 — Phase 4: Transaction Service

PHASE 4

Deposits, Withdrawals, Transfers — The Core Ledger

Weeks 10–13

What to Build

- Deposit — credit an account, record transaction
- Withdrawal — check balance, debit account, record transaction
- Transfer — atomic debit from source, credit to destination (same or different customer)
- Transaction history with pagination, date filter, type filter
- Reference number generation: TXN + YYYYMMDD + 8 random chars
- Concurrency protection: database-level locking for balance updates
- Reversal mechanism — when a completed transaction must be undone

Atomic Transfer — Most Critical Code

```
// app/Services/TransferService.php
public function transfer(TransferDTO $dto): array
{
    return DB::transaction(function () use ($dto) {
        // Lock both accounts for update — prevents concurrent balance corruption
        $source = Account::lockForUpdate()->findOrFail($dto->sourceAccountId);
        $dest   = Account::lockForUpdate()->findOrFail($dto-
>destinationAccountId);
```

```
        if ($source->balance < $dto->amount) {
            throw new InsufficientBalanceException(
                available: $source->balance,
                requested: $dto->amount
            );
        }

        if ($source->status !== AccountStatus::Active) {
            throw new AccountNotActiveException($source->account_number);
        }

        // Debit source
        $debitTxn = $this->createTransaction($source, TransactionType::Debit,
$dto);

        // Credit destination
        $creditTxn = $this->createTransaction($dest, TransactionType::Credit,
$dto,
                                              relatedTxnId: $debitTxn->id);

        // Update debit with related credit ID
        $debitTxn->update(['related_txn_id' => $creditTxn->id]);

        // Update balances via Account Service API
        $this->accountService->updateBalance($source->id, -$dto->amount);
        $this->accountService->updateBalance($dest->id, $dto->amount);

        return [$debitTxn, $creditTxn];
    });
}
```

Phase 4 Done When

Deposit ₹10,000 → balance shows ₹10,000, one credit transaction in history
Withdraw ₹3,000 → balance shows ₹7,000, one debit transaction in history
Transfer ₹2,000 to another account → source shows ₹5,000, dest shows ₹2,000
Withdraw ₹6,000 from ₹5,000 account → 422 with INSUFFICIENT_BALANCE error
Transactions table: confirm no UPDATE or DELETE routes exist in the service
All 3 operations appear correctly in paginated GET history

Section 13 — Phase 5: Payment Service & Razorpay

PHASE 5

Razorpay Full Integration — Orders, Webhooks, Refunds, Payouts

Weeks 14–17

What to Build

- Razorpay order creation (pay-in flow)
- Payment verification after checkout (HMAC signature check)
- Webhook receiver with signature verification — queued processing
- Refund initiation via Razorpay API
- Payout to bank account via Razorpay Payout API (sandbox)
- Every successful payment triggers a transaction in transaction-service
- Webhook audit log — every webhook received is stored before processing

Razorpay Setup

```
// config/services.php
'razorpay' => [
    'key_id'         => env('RAZORPAY_KEY_ID'),
    'key_secret'     => env('RAZORPAY_KEY_SECRET'),
    'webhook_secret' => env('RAZORPAY_WEBHOOK_SECRET'),
],

// composer.json dependency
"razorpay/razorpay": "^2.8"

// .env (use TEST keys from Razorpay dashboard)
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxxxx
RAZORPAY_WEBHOOK_SECRET=your_webhook_secret
```

Create Order + Verify Payment Flow

```
// Step 1: Frontend calls POST /api/v1/payments/create-order
// Step 2: Payment Service creates Razorpay order, saves to DB
// Step 3: Frontend opens Razorpay checkout with order_id
// Step 4: Customer pays in Razorpay sandbox
// Step 5: Razorpay calls webhook + frontend gets payment response
// Step 6: Frontend calls POST /api/v1/payments/verify with
//         { razorpay_order_id, razorpay_payment_id, razorpay_signature }
// Step 7: Payment Service verifies signature:
//         expected = HMAC(order_id + "|" + payment_id, key_secret)
// Step 8: If valid → update order status → call transaction-service to record

// Signature verification
public function verifyPayment(VerifyPaymentRequest $request): JsonResponse
{
    $expected = hash_hmac(
        "sha256",
        $request->razorpay_order_id . "|" . $request->razorpay_payment_id,
        config('services.razorpay.key_secret')
    );
```

```
    if (! hash_equals($expected, $request->razorpay_signature)) {
        return $this->error("PAYMENT_VERIFICATION_FAILED", "Signature mismatch",
400);
    }

    // Update order, notify transaction service
    ...
}
```

Phase 5 Done When

Create a Razorpay test order and complete payment in sandbox
POST /payments/verify returns success with valid signature, 400 with tampered data
Webhook received, signature verified, logged to webhook_logs table
Successful payment creates a transaction record in transaction-service
Refund initiated via /payments/refund and visible in Razorpay test dashboard
Payout created via /payments/payout in Razorpay sandbox

Section 14 — Phase 6: Loan Service

PHASE 6

Loan Lifecycle — Application, Approval, EMI, Repayment

Weeks 18–22

What to Build

- Loan application submission (checks KYC + active account)
- Officer review workflow: approve or reject with reason
- EMI schedule generation on approval using reducing balance method
- Disbursement: transfers loan amount to customer account via transaction-service
- EMI repayment recording
- Foreclosure calculation (remaining outstanding with interest adjustment)
- Overdue detection: flag EMIs past due date

EMI Calculation — Reducing Balance Method

```
// app/Services/EmiCalculator.php
class EmiCalculator
{
    /**
```

```
     * @return array  Monthly EMI amount (same every month)
     */
    public function monthlyEmi(float $principal, float $annualRate, int $months):
float
    {
        $r = ($annualRate / 100) / 12;  // monthly rate

        if ($r === 0.0) {
            return round($principal / $months, 2);
        }

        $emi = $principal * $r * pow(1 + $r, $months)
                         / (pow(1 + $r, $months) - 1);

        return round($emi, 2);
    }

    /**
     * Generate full EMI schedule
     * @return array<int, array{emi_number, due_date, emi_amount,
     *                          principal, interest, outstanding}>
     */
    public function generateSchedule(
        float $principal, float $annualRate, int $months, Carbon $startDate
    ): array {
        $emi     = $this->monthlyEmi($principal, $annualRate, $months);
        $r       = ($annualRate / 100) / 12;
        $balance = $principal;
        $schedule = [];

        for ($i = 1; $i <= $months; $i++) {
            $interest  = round($balance * $r, 2);
            $principal = round($emi - $interest, 2);
            $balance   = round($balance - $principal, 2);

            $schedule[] = [
                'emi_number'  => $i,
                'due_date'    => $startDate->copy()->addMonths($i)->format('Y-m-
d'),
                'emi_amount'  => $emi,
                'principal'   => $principal,
                'interest'    => $interest,
                'outstanding' => max($balance, 0),
            ];
        }

        return $schedule;
    }
}
```

Phase 6 Done When

Customer submits loan application — only allowed if KYC approved + active account

Officer approves → EMI schedule generated and stored for all months
Disbursement: loan amount appears as credit transaction in customer account
EMI repayment via /loans/{id}/repay records payment, marks EMI as paid
Overdue EMIs have status: overdue when past due_date and unpaid
Foreclosure returns correct outstanding amount

Section 15 — Phase 7: Frontend (Blade + Livewire)

PHASE 7

Full UI — Customer, Officer & Admin Dashboards

Weeks 23–27

Frontend Architecture Decision

The frontend lives inside the api-gateway service as Blade views. It makes API calls to the internal services via the same HTTP layer. You are NOT building a separate SPA — Livewire handles reactive UI within Blade, which is the right choice for a solo developer who already knows PHP.

Three Dashboard Types

Dashboard	Users	Key Screens
Customer Portal	Bank customers	Account summary, transaction history, KYC upload, loan apply, Razorpay checkout
Officer Panel	Bank staff	KYC review queue, loan application queue, account management, customer search
Admin Panel	System admin	User management, all audit logs, system health, service status

Tech Setup for Frontend

```
# In api-gateway service
composer require livewire/livewire
npm install -D tailwindcss @tailwindcss/forms @tailwindcss/typography
npx tailwindcss init

# Install Alpine.js for lightweight JS interactions
npm install alpinejs

# Install Razorpay JS SDK (add to blade layout)
```

```
# <script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

Key Livewire Components to Build

Component	File	Functionality	Complexity
AccountSummary	Livewire/AccountSummary.php	Real-time balance display	Low
TransactionHistory	Livewire/TransactionHistory.php	Paginated, filterable list	Medium
KycUploader	Livewire/KycUploader.php	Multi-file upload with progress	Medium
KycReviewQueue	Livewire/KycReviewQueue.php	Officer KYC approve/reject	Medium
LoanApplication	Livewire/LoanApplication.php	Multi-step loan form	High
LoanEmiSchedule	Livewire/LoanEmiSchedule.php	EMI table with status	Low
PaymentCheckout	Livewire/PaymentCheckout.php	Razorpay JS integration	High
AuditLogViewer	Livewire/AuditLogViewer.php	Admin audit log table	Medium

Phase 7 Done When

Customer can login, see accounts, view last 10 transactions, initiate a payment
Customer can upload KYC documents through the UI
Officer can see KYC queue, click approve/reject with remarks
Customer can apply for a loan and see the EMI schedule after approval
Razorpay checkout opens in browser, test payment completes, transaction appears
Admin can view audit logs filtered by date and action type

Section 16 — Phase 8: Testing, Docs & Deployment

PHASE 8

Polish — Tests, OpenAPI Docs, VPS Deployment
Weeks 28–30

Testing Strategy

Test Type	Tool	What to Test
Unit Tests	Pest	EmiCalculator, AccountNumberGenerator, KYC state transitions
Feature Tests	Pest + Laravel HTTP	Every API endpoint: happy path + error cases

Integration	Pest + real DB	Full transaction flow, loan disbursement
Security Tests	Pest	Webhook signature rejection, unauthenticated access

Critical Tests to Write

```
// tests/Feature/TransactionTest.php
it('prevents overdraft', function () {
    $account = Account::factory()->create(['balance' => 500.00]);

    $response = $this->postJson('/api/v1/transactions/withdraw', [
        'account_id' => $account->id,
        'amount'     => 1000.00,
    ]);

    $response->assertStatus(422)
             ->assertJson(['error' => ['code' => 'INSUFFICIENT_BALANCE']]);

    expect($account->fresh()->balance)->toBe(500.00);  // unchanged
});

it('rejects webhook with invalid signature', function () {
    $response = $this->postJson('/api/v1/payments/webhook',
        ['event' => 'payment.captured'],
        ['X-Razorpay-Signature' => 'fakesignature']
    );
    $response->assertStatus(401);
});
```

Deployment — DigitalOcean $6/month Droplet

```
# 1. Create Ubuntu 22.04 Droplet on DigitalOcean ($6/mo, 1GB RAM)
# 2. Install Docker + Docker Compose on the server
# 3. Point a domain to the Droplet IP (or use the IP directly)
# 4. Clone your repo: git clone https://github.com/you/bankcore.git
# 5. Copy .env files and set production values
# 6. Get free SSL cert:
     docker run -it --rm -p 80:80 certbot/certbot certonly --standalone -d
yourdomain.com
# 7. Start everything:
     docker compose up -d
# 8. Run migrations for all services:
     docker compose exec auth-service php artisan migrate
     docker compose exec customer-service php artisan migrate
     # ... repeat for each service
# 9. Run seeders:
     docker compose exec auth-service php artisan db:seed
```

Phase 8 Done When (Project Complete)

70%+ test coverage on critical paths (transactions, payments, loans)
All services have OpenAPI docs auto-generated via Scramble
Project is deployed and accessible via a public URL
README.md explains architecture, setup, and all test credentials
Postman collection exported and added to docs/
Someone else (friend, AI reviewer) can clone, run, and understand the project

Section 17 — Timeline & Effort Breakdown

Phase	Focus	Weeks	Hardest Part
0 — Foundation	Docker + nginx + service skeletons	1–2	docker-compose networking
1 — Auth	JWT tokens, roles, middleware	3–4	Token refresh flow cross-service
2 — Customer/KYC	Profiles, file upload, encryption	5–7	State machine + audit trail
3 — Accounts	Account CRUD, cross-service KYC check	8–9	Inter-service HTTP calls
4 — Transactions	Ledger, concurrency, immutability	10–13	Atomic transfer with DB locks
5 — Payments	Razorpay full flow + webhooks	14–17	Webhook verification + queues
6 — Loans	EMI math, state machine, disbursement	18–22	Reducing balance EMI schedule
7 — Frontend	Livewire dashboards, Razorpay checkout	23–27	Livewire + Razorpay JS bridge
8 — Polish	Tests, docs, VPS deployment	28–30	Writing tests after the fact

Realistic Expectation

Total: ~30 weeks at 10-15 hours per week = roughly 7 months
This is NOT a weekend project. It is a career-defining portfolio piece.
Each phase should be fully working before the next one starts.
Do NOT rush. A solid Phase 1 makes every phase after it 3x easier.
If you get stuck, build the monolith version first then extract services — that is fine.

Section 18 — Decision Log

Decision	Alternatives Considered	Why This Was Chosen
Monorepo	Polyrepo (7 separate Git repos)	Solo dev needs one place to see everything; easier navigation, shared docs
Blade + Livewire	React/Vue SPA	No context switch; stay in PHP world; faster to build alone
MySQL per service	Shared MySQL, PostgreSQL	Microservices isolation principle; familiar to developer
REST APIs	GraphQL, gRPC	Simpler, well-documented, matches Laravel conventions

Redis queues	RabbitMQ, Kafka	Already in Laravel ecosystem; Kafka is overkill for this scale
Razorpay	PayU, Cashfree, Stripe	Best PHP SDK quality, best sandbox, most Indian tutorials available
Sanctum + JWT	Passport (OAuth2)	Sanctum is simpler and sufficient; Passport adds unnecessary complexity
PHP 8.3 Enums	String constants, integer codes	Type-safe, IDE-friendly, match expression compatible
Pest	PHPUnit directly	Cleaner syntax, Laravel native, easier to write and read
Docker Compose	Laravel Sail, Homestead	Direct Docker knowledge is more transferable to real jobs

Section 19 — Learning Resources & Next Steps

Must-Read Before Each Phase

Phase	Topic	Resource	Why
0	Docker Compose networking	docs.docker.com/compose/networking	Services talk by name, not IP
1	Laravel Sanctum	laravel.com/docs/sanctum	Official auth docs
2	Laravel File Storage	laravel.com/docs/filesystem	Secure file handling
4	Database transactions	laravel.com/docs/database#database-transactions	Atomic operations
5	Razorpay PHP SDK	razorpay.com/docs/server-integration/php	Official Razorpay PHP guide
5	Razorpay Webhooks	razorpay.com/docs/webhooks	Webhook events list
6	Reducing balance EMI	Any Indian banking textbook	EMI formula derivation
7	Livewire v3	livewire.laravel.com	Official Livewire docs
8	Pest testing	pestphp.com/docs	Test writing guide

After This Project — What to Learn Next

- Event Sourcing — your immutable transaction ledger is already half-way there
- CQRS — separate read models from write models for reporting performance
- API Gateway patterns — rate limiting, circuit breakers, service discovery
- Message brokers — replace Redis queues with RabbitMQ for real enterprise scale

- OpenBanking APIs — AA (Account Aggregator) framework, OCEN protocol
- Real Aadhaar eKYC — UIDAI sandbox registration and API integration

Final Note — Start Today

The biggest mistake is waiting until you fully understand everything before starting.
You will learn Docker by fighting with docker-compose.yml in Phase 0.
You will learn microservices by making your first cross-service HTTP call in Phase 3.
You will learn banking by implementing the transfer atomic lock in Phase 4.
Start with: mkdir bankcore && git init
Everything else follows from that first command.

You have the plan.

Now go build it.

BankCore — Laravel 11 | PHP 8.3 | Microservices | Razorpay